

# Scaling Book — Section 9 Exercises

Rahul Bir

## Q projection op exercise

$X[32, 1024, 8192] \cdot W_Q[8192, 32, 256]$  on v2  $4 \times 2$  slice.

$$\text{BS} = \frac{32 \cdot 1024}{4} = 8192 \quad \text{so we're compute bound.}$$

$$\text{FLOPS} = 2 \cdot 32 \cdot 1024 \cdot 8192 \cdot 32 \cdot 256 \approx 2.46 \cdot 10^{12}.$$

$$T_{\text{comp}} = \frac{2.46 \cdot 10^{12}}{8 \cdot 23 \cdot 10^{12}} = 13.4 \text{ ms.}$$

This is close to the wall duration in the profile!

## Worked Problems

### Q1) Trace

**multiply\_reduce\_fusion.1:**

$$\underbrace{\text{bf16}[4096, 8192]}_{W_1} \cdot \underbrace{\text{bf16}[8192]}_{\text{In}} \rightarrow \underbrace{\text{bf16}[4096]}_{\text{Tmp}} \Rightarrow \text{matmul}$$

**multiply\_reduce\_fusion:**

$$\underbrace{\text{bf16}[8192, 4096]}_{W_2} \cdot \underbrace{\text{bf16}[4096]}_{\text{Tmp}} \rightarrow \underbrace{\text{bf16}[8192]}_{\text{Out}} \Rightarrow \text{matmul (feeding in result from last HLO op)}$$

**all\_reduce:**

$$\text{all\_reduce}(\text{bf16}[1, 8192]) \rightarrow \text{bf16}[1, 8192]$$

Running on a v5e-8 slice and replica groups is 8, so sharded across all devices on contracting dim.

### Computation:

$$\begin{aligned} W_1[I_{xy}, J] \cdot_J \text{In}[J] &\rightarrow \text{Tmp}[I_{xy}] \\ W_2[K, I_{xy}] \cdot_I \text{Tmp}[I_{xy}] &\rightarrow \text{Out}[K]\{U_{xy}\} \\ \text{AllReduce}_{xy}(\text{Out}[K]\{U_{xy}\}) &\rightarrow \text{Out}[K] \end{aligned}$$

$$\underbrace{\text{bf16}[32768, 8192]}_{W_1} \cdot \underbrace{\text{bf16}[8192]}_{\text{In}} \cdot \underbrace{\text{bf16}[8192, 32768]}_{W_2} \rightarrow \text{bf16}[8192]$$

This operation is essentially the up and down projection in an MLP where the  $F$  dim is sharded across all 8 devices.

## Q2) Profiling & Sharding Optimization

The following is the full Jupyter notebook used to profile and re-shard the forward pass. Each code block corresponds to one cell in the notebook.

### Cell 1: Imports, Config, Layer, Weights

```
1 from flax import struct
2 from typing import Callable, Tuple
3
4 import collections
5
6 import jax
7 import jax.numpy as jnp
8
9 P = jax.sharding.PartitionSpec
10
11 ShardingRules = collections.namedtuple('ShardingRules',
12   ['batch', 'sequence', 'd_model', 'query_heads', 'key_heads', 'key_dim', 'd_ff', 'vocab'])
13
14 def _logical_to_physical(logical: Tuple[str, ...], rules: ShardingRules):
15     """Converts logical to physical pspec."""
16     return P(*(getattr(rules, l) for l in logical))
17
18 def _logical_to_sharding(logical: Tuple[str, ...], mesh: jax.sharding.Mesh, rules: ShardingRules):
19     """Converts logical to sharding."""
20     return jax.sharding.NamedSharding(mesh, _logical_to_physical(logical, rules))
21
22
23 @struct.dataclass
24 class Config:
25     d_model: int
26     ffw_multiplier: int
27     num_layers: int
28
29     query_heads: int
30     kv_heads: int
31     key_dim: int
32
33     vocab_size: int
```

```

34
35     dtype: jnp.dtype = jnp.bfloat16
36
37
38 @struct.dataclass
39 class TensorInfo:
40     shape: jax.ShapeDtypeStruct
41     logical_axes: tuple[str, ...]
42     initializer: Callable | None = None
43
44
45 @struct.dataclass
46 class Layer:
47
48     q: jax.Array | TensorInfo
49     k: jax.Array | TensorInfo
50     v: jax.Array | TensorInfo
51     proj: jax.Array | TensorInfo
52
53     w1: jax.Array | TensorInfo
54     w2: jax.Array | TensorInfo
55
56     gamma1: jax.Array | TensorInfo
57     gamma2: jax.Array | TensorInfo
58
59 @classmethod
60 def abstract(cls, cfg: Config):
61     return Layer(
62         q=TensorInfo(
63             jax.ShapeDtypeStruct((cfg.d_model, cfg.query_heads, cfg.key_dim),
64                                   dtype=cfg.dtype),
65             ('d_model', 'query_heads', 'key_dim'),
66             jax.nn.initializers.he_normal(in_axis=0, out_axis=(1,2))),
67         k=TensorInfo(
68             jax.ShapeDtypeStruct((cfg.d_model, cfg.kv_heads, cfg.key_dim), dtype=
69                                   cfg.dtype),
70             ('d_model', 'query_heads', 'key_dim'),
71             jax.nn.initializers.he_normal(in_axis=0, out_axis=(1,2))),
72         v=TensorInfo(
73             jax.ShapeDtypeStruct((cfg.d_model, cfg.kv_heads, cfg.key_dim), dtype=
74                                   cfg.dtype),
75             ('d_model', 'query_heads', 'key_dim'),
76             jax.nn.initializers.he_normal(in_axis=0, out_axis=(1,2))),
77         proj=TensorInfo(
78             jax.ShapeDtypeStruct((cfg.query_heads, cfg.key_dim, cfg.d_model),
79                                   dtype=cfg.dtype),
80             ('query_heads', 'key_dim', 'd_model'),
81             jax.nn.initializers.he_normal(in_axis=(0,1), out_axis=2)),
82         w1=TensorInfo(
83             jax.ShapeDtypeStruct((cfg.d_model, cfg.d_model * cfg.ffw_multiplier),
84                                   dtype=cfg.dtype),
85             ('d_model', 'd_ff'),
86             jax.nn.initializers.he_normal(in_axis=0, out_axis=1)),
87         w2=TensorInfo(

```

```

88         jax.ShapeDtypeStruct((cfg.d_model * cfg.ffw_multiplier, cfg.d_model),
89                               dtype=cfg.dtype),
90         ('d_ff', 'd_model'),
91         jax.nn.initializers.he_normal(in_axis=0, out_axis=1),
92     ),
93     gamma1=TensorInfo(
94         jax.ShapeDtypeStruct((cfg.d_model,), dtype=cfg.dtype),
95         ('d_model',),
96         jax.nn.initializers.constant(1.0),
97     ),
98     gamma2=TensorInfo(
99         jax.ShapeDtypeStruct((cfg.d_model,), dtype=cfg.dtype),
100        ('d_model',),
101        jax.nn.initializers.constant(1.0),
102    ),
103
104 @struct.dataclass
105 class Weights:
106     layers: list[Layer]
107     embedding: jax.Array | TensorInfo
108
109     @classmethod
110     def abstract(cls, cfg: Config):
111         return Weights(
112             layers=[Layer.abstract(cfg) for _ in range(cfg.num_layers)],
113             embedding=TensorInfo(
114                 jax.ShapeDtypeStruct((cfg.vocab_size, cfg.d_model), cfg.dtype),
115                 ('vocab', 'd_model'),
116                 jax.nn.initializers.he_normal(in_axis=0, out_axis=1)
117             ),
118         )
119
120     @classmethod
121     def sharding(cls, cfg: Config, mesh: jax.sharding.Mesh, rules: ShardingRules):
122         abstract = cls.abstract(cfg)
123         return jax.tree.map(lambda info: _logical_to_sharding(info.logical_axes, mesh,
124                                                                rules), abstract,
125                             is_leaf=lambda x: isinstance(x, TensorInfo))
126
127     @classmethod
128     def init(cls, cfg: Config, key: jax.random.PRNGKey, mesh: jax.sharding.Mesh,
129              rules: ShardingRules):
130         abstract = cls.abstract(cfg)
131
132         def _init():
133             num_leaves = len(jax.tree.leaves(abstract, is_leaf=lambda x: isinstance(x,
134                                                                                       TensorInfo)))
135             rng_iter = iter(jax.random.split(key, num_leaves))
136             return jax.tree.map(
137                 lambda info: info.initializer(next(rng_iter), info.shape.shape, info.
138                                                 shape.dtype), abstract,
139                 is_leaf=lambda x: isinstance(x, TensorInfo))
140
141         sharding = cls.sharding(cfg, mesh, rules)
142         return jax.jit(_init, out_shardings=sharding)()

```

## Cell 2: rms\_norm, layer\_forward, forward

```

1 def rms_norm(x: jax.Array, gamma: jax.Array) -> jax.Array:
2     """Apply RMS normalization."""
3     """
4     ORIGINAL:
5     Negligible flops but for sharding scheme:
6     x[B, T, D@X] => x[B, T] by fused vector ops in the last matmul and then an AR
7         across the partial sums for the mean
8     outputs similar sharding x[B, T, D@X]
9
10    INSTEAD:
11    Still need to AR but now that B is sharded as well, this comm should be faster
12    """
13    rms = jax.lax.rsqrt(jnp.mean(jnp.square(x), axis=-1, keepdims=True) + 1e-6)
14    return gamma * x * rms
15
16 def layer_forward(x: jax.Array, layer: Layer) -> jax.Array:
17     x = jax.lax.with_sharding_constraint(x, jax.P('x', None, 'y'))
18
19     with jax.named_scope('pre_attn_norm'):
20         attn_in = rms_norm(x, layer.gamma1)
21
22     # Q, K, V projections
23     with jax.named_scope('qkv_matmul'):
24         """
25         ORIGINAL:
26         roofline = 3*2*btdhn / (8 * 1.97e14) = 1ms
27         actual = 8.174ms
28         Note: (N == Q == K == V)
29
30         op: x[B, T, D@X] @ W_q[D@X, N@Y, H] => q,k,v[N@Y, H, B, T]
31         steps:
32         x[B, T, D@X] @ W_q[D@X, N@Y, H] => q,k,v[N@Y, H, B, T]{U_X}
33         AR_X(q,k,v[N@Y, H, B, T]{U_X}) = q,k,v[N@Y, H, B, T]
34         (AR dominates the time here)
35
36         INSTEAD:
37         If x[B@X, T, D@Y] (like in the mixed FSDP+TP section):
38         x[B@X, T, D@Y] @ W_q[D@X, N@Y, H] => x[B@X, T, N@Y, H]
39         (still need to do 2 AG which is still slightly less than AR since
40         one AG is over Y while the AR is over X. Additionally, one AG
41         can be done ahead of time if optimized by the compiled.)
42         """
43         q = jnp.einsum('btd,dhq->bhtq', attn_in, layer.q)
44         k = jnp.einsum('btd,dhk->bhtk', attn_in, layer.k)
45         v = jnp.einsum('btd,dhv->bhtv', attn_in, layer.v)
46
47     # TODO: add RoPE here
48
49     # just to ensure
50     x = jax.lax.with_sharding_constraint(x, jax.P('x', None, 'y'))
51
52     # Attention
53     with jax.named_scope('attn'):
54         """
55         ORIGINAL:
56         roofline = (2bhstn + 2bhstn) / (8 * 1.97e14) = 0.08ms

```

```

56     actual = 1ms
57
58     (already very small, not dominant flops)
59     """
60     scale = q.shape[-1] ** -0.5
61     # TODO: add masking here
62
63     num_query_heads, num_kv_heads = q.shape[1], k.shape[1]
64
65     if num_query_heads == num_kv_heads or num_kv_heads == 1:
66         qk = jnp.einsum('bhtd,bhsd->bhts', q, k) * scale
67         logits = jax.nn.softmax(qk.astype(jnp.float32), axis=-1)
68         attn_vec = jnp.einsum('bhsd,bhts->bhtd', v, logits)
69     else:
70         assert num_query_heads % num_kv_heads == 0
71         q = q.reshape(q.shape[0:1] +
72                       (num_kv_heads, num_query_heads // num_kv_heads) +
73                       q.shape[2:])
74         qk = jnp.einsum('bqhtd,bhsd->bqhts', q, k) * scale
75         logits = jax.nn.softmax(qk.astype(jnp.float32), axis=-1)
76         attn_vec = jnp.einsum('bqsd,bqhts->bqhtd', v, logits)
77         attn_vec = attn_vec.reshape(attn_vec.shape[0:1] + (num_query_heads,) +
78                                   attn_vec.shape[3:])
79
80     # Attention proj
81     """
82     ORIGINAL:
83     roofline = 2bhtnd / (8 * 1.97e14) = 0.3ms
84     actual = 1.98ms
85     (dominated by AR)
86     op: attn_vec[B, H, T, N@Y] @ [H, N@Y, D@X] => [B, T, D@X]
87     steps:
88     attn_vec[B, H, T, N@Y] @ [H, N@Y, D@X] => [B, T, D@X]{U_Y}
89     AR_Y([B, T, D@X]{U_Y}) => [B, T, D@X]
90
91     INSTEAD:
92     op: attn_vec[B@X, H, T, N@Y] @ [H, N@Y, D@X] => [B@X, T, D@Y]
93     steps:
94     AG_X([H, N@Y, D@X]) => [H, N@Y, D]
95     attn_vec[B@X, H, T, N@Y] @ [H, N@Y, D] => [B@X, T, D]{U_Y}
96     RS_Y,D([B@X, T, D]{U_Y}) => [B@X, T, D@Y]
97
98     Timing doesn't change that much under sharding.
99     """
100     with jax.named_scope('attn_proj'):
101         attn_out = jnp.einsum('bhtv,hvd->btd', attn_vec, layer.proj)
102
103     # Residual connection
104     with jax.named_scope('residual'):
105         x = x + attn_out
106
107     # Second RMSNorm (Pre-LN for FFN)
108     with jax.named_scope('ffn_pre_norm'):
109         ffw_in = rms_norm(x, layer.gamma2)
110
111     jax.lax.with_sharding_constraint(ffw_in, jax.P('x', None, 'y'))
112
113     """
114     ORIGINAL:

```

```

114     roofline = 2*2btdd / (8 * 1.97e14) = 5ms
115     actual = 24.9ms (much larger!)
116
117     op1: [B, T, D@X] @ [D@X, F@Y] = [B, T, F@Y]
118     steps:
119     [B, T, D@X] @ [D@X, F@Y] = [B, T, F@Y]{U_X}
120     AR_X([B, T, F@Y]{U_X}) = [B, T, F@Y]
121
122     op2: [B, T, F@Y] @ [F@Y, D@X] => [B, T, D@X]
123     steps:
124     [B, T, F@Y] @ [F@Y, D@X] => [B, T, D@X]{U_Y}
125     AR_Y([B, T, D@X]{U_Y}) = [B, T, D@X]
126
127     INSTEAD:
128     op1: [B@X, T, D@Y] @ [D@X, F@Y] => [B@X, T, F@Y]
129     steps:
130     AG_Y([B@X, T, D@Y]) = [B@X, T, D]
131     AG_X([D@X, F@Y]) = [D, F@Y]
132     [B@X, T, D] @ [D, F@Y] = [B@X, T, F@Y]
133
134     op2: [B@X, T, F@Y] @ [F@Y, D@X] => [B@X, T, D@Y]
135     steps:
136     AG_X([F@Y, D@X]) = [F@Y, D]
137     [B@X, T, F@Y] @ [F@Y, D] => [B@X, T, D]{U_Y}
138     RS_Y,D([B@X, T, D]{U_Y}) = [B@X, T, D@Y]
139
140     (under the new scheme, we would have cheaper comms)
141     """
142     with jax.named_scope('ffw'):
143         ffw_out = jnp.einsum('btd,df->btff', ffw_in, layer.w1).astype(jnp.bfloat16)
144         ffw_out = jax.nn.gelu(ffw_out)
145         ffw_out = jnp.einsum('btff,fd->btd', ffw_out, layer.w2).astype(jnp.bfloat16)
146
147     # Residual connection
148     with jax.named_scope('residual'):
149         x = x + ffw_out
150
151     # just to ensure
152     x = jax.lax.with_sharding_constraint(x, jax.P('x', None, 'y'))
153
154     return x
155
156
157 def forward(x: jax.Array, weights: Weights) -> jax.Array:
158     """Forward pass through the network."""
159
160
161     # Initially, x = [B@X, T]
162     one_hot = jax.nn.one_hot(x, cfg.vocab_size)
163     """
164     ORIGINAL
165     x = [B@X, T, V]
166     bottom matmul should take (2btvd) / (8 * 1.97e14) = 2.7ms
167     actual time = 11.4ms!
168
169     op: x[B@X, T, V] @ [V, D@X] => [B, T, D@X]
170     steps:
171     AG_X(x[B@X, T, V]) => x[B, T, V]
172     x[B, T, V] @ [V, D@X] => [B, T, D@X]

```

```

173
174
175 INSTEAD
176 op: x[B@X, T, V] @ [V, D@X] => [B@X, T, D@Y]
177 steps:
178 Reshard([V, D@X]) => [V, D@Y]
179 x[B@X, T, V] @ [V, D@Y] => [B@X, T, D@Y]
180 """
181 x = jnp.einsum('vd,btv->btd', weights.embedding, one_hot)
182
183 for idx, layer in enumerate(weights.layers):
184     with jax.named_scope(f'layer_{idx}'):
185         x = layer_forward(x, layer)
186
187 logits = jnp.einsum('vd,btd->btv', weights.embedding, x)
188 return jax.nn.log_softmax(logits, axis=-1)

```

### Cell 3: Sharding rules

```

1 fsdp_rules = ShardingRules(
2     batch=('x', 'y'),
3     sequence=None,
4     d_model=('x', 'y'),
5     query_heads=None,
6     key_heads=None,
7     key_dim=None,
8     d_ff=None,
9     vocab=None
10 )
11
12 model_parallel_rules = ShardingRules(
13     batch=None,
14     sequence=None,
15     d_model=None,
16     query_heads=('x', 'y'),
17     key_heads=('x', 'y'),
18     key_dim=None,
19     d_ff=('x', 'y'),
20     vocab=None
21 )
22
23 mixed_rules = ShardingRules(
24     batch='x',
25     sequence=None,
26     d_model='x',
27     query_heads='y',
28     key_heads='y',
29     key_dim=None,
30     d_ff='y',
31     vocab=None
32 )

```

### Cell 4: Clear live arrays

```

1 for arr in jax.live_arrays():
2     arr.delete()

```

## Cell 5: Config, mesh, init weights

```
1 Auto = jax.sharding.AxisType.Auto
2
3 cfg = Config(
4     d_model=8192,
5     ffw_multiplier=4,
6     num_layers=4,
7     query_heads=16,
8     kv_heads=16,
9     key_dim=256,
10    vocab_size=32_128,
11    dtype=jnp.bfloat16,
12 )
13
14 mesh = jax.make_mesh((4, 2), ('x', 'y'), (Auto, Auto))
15 jax.set_mesh(mesh)
16 print(mesh)
17
18 rules = mixed_rules
19
20 rng = jax.random.PRNGKey(42)
21 weights = Weights.init(cfg, rng, mesh, rules)
```

## Cell 6: Compile

```
1 batch_size = 8
2 seq_len = 1024
3
4 input_sharding = _logical_to_sharding(('batch', 'sequence'), mesh=mesh, rules=
    rules)
5 x = jnp.zeros((batch_size, seq_len), dtype=jnp.int32, device=input_sharding)
6
7 compiled = jax.jit(forward).lower(x, weights).compile()
```

## Cell 7: Profile

```
1 with jax.profiler.trace("/kaggle/working/tensorboard"):
2     logits = compiled(x, weights)
3     jax.block_until_ready(logits)
```

## Cell 8: Zip the tensorboard trace

```
1 import os
2 import shutil
3 from IPython.display import FileLink, display
4
5 # Folder where JAX wrote the TensorBoard trace
6 src_dir = "/kaggle/working/tensorboard"
7
8 # Zip output path
9 zip_base = "/kaggle/working/tensorboard"
10 zip_file = zip_base + ".zip"
11
12 # Check trace folder exists
13 if not os.path.exists(src_dir):
```

```

14     raise FileNotFoundError(f"{src_dir} does not exist. Make sure your profiler
15         wrote here.")
16
17 # Remove old zip if present
18 if os.path.exists(zip_file):
19     os.remove(zip_file)
20
21 # Create zip
22 shutil.make_archive(
23     base_name=zip_base,
24     format="zip",
25     root_dir=src_dir,
26 )
27
28 # Verify it exists
29 print("Created:", zip_file)
30 !ls -lh /kaggle/working/tensorboard.zip
31
32 # Make Kaggle download link
33 os.chdir("/kaggle/working")
34 display(FileLink("tensorboard.zip"))

```

## Additional Questions

Under this fix, the highest MXU I now get is 12.7% compared to 13.3%. However, the total forward pass time has halved from 206 ms to 109 ms. I suspect the slight decrease in MXU might be from more comms launched even though each comm is much cheaper.

1. The sharding strategy I changed it to is a mixed FSDP + TP. Specifically, this is 4-way FSDP and 2-way TP.
2.  $BS = 8$  but per device it's  $BS_{\text{local}} = 2$ .  $d_{\text{model}} = 8192$  but per device (on activations) becomes 2048.  $d_{\text{ff}} = 4 \cdot d_{\text{model}} = 32768$  but per device is 16384.
3. Including the projections in attention, under my optimized scheme, the attention block takes around 7 ms while the MLP block takes around 11 ms.
4. Rooflines are written above their corresponding code blocks above (BS and sequence length are large enough that on roofline we should be compute bound).